

Hardest SQL Questions with Expected Outputs

1. Longest streak of consecutive order days (per customer)

CustomerID	Longest_Streak_Days
C101	5
C102	3

```
select customer_id, max(streak_length) as 'max_streak' from (select customer_id,
order_date,

@counter := if(customer_id = @prev_id and datediff(order_date, @prev_date) = 1,
@counter + 1, 1) as 'streak_length',

@prev_id := customer_id,

@prev_date := order_date

from (select distinct * from (select customer_id, order_date from cleanedsup) t) m,

(select @prev_id := null, @prev_date := null, @counter:=0) as init

order by customer_id, order_date) z

group by customer_id
```

2. 7-day rolling average revenue per day

OrderDate	Total_Revenue	Rolling_7Day_Avg
2024-01-01	1000	1000
2024-01-02	1200	1100

```
select order_date,

sum(sales) over(partition by order_date) as 'total_revenue',

avg(sales) over(partition by order_date order by order_date rows between 6 preceding and
current row) as 'rolling_avg'

from cleanedsup
```

3. Customers who bought in 3+ consecutive months

CustomerID	First_Month	Last_Month	Streak_Months
C105	2023-01	2023-03	3

NOTE:- when you want to calculate the diff between dates in months, use `period_diff` because `datediff` only gives diff in days. most importantly, `period_diff()` expects the dates to be in format `%Y%m` and NOT `%Y-%m` so do remember that.

```
select customer_id, max(streak_length) as 'max_streak', fv as 'first_month', lv as 'last_month'
from (select customer_id, streak_length,

first_value(months) over(partition by customer_id order by months) as 'fv',

first_value(months) over(partition by customer_id order by months desc) as 'lv'

from (select customer_id, months,

@counter:= if(customer_id = @prev_id and period_diff(months, @prev_month) = 1,

@counter + 1, 1) as 'streak_length',

@prev_id := customer_id as 'updated_id',

@prev_month := months as 'updated_month'

from (select distinct * from (select customer_id, date_format(order_date, "%Y%m")

as 'months' from cleanedsup) t) m , (select @prev_id := null, @prev_month := null, @counter

:= 0) as init

order by customer_id, months) z) k

group by customer_id, fv, lv
```

4. Monthly revenue growth rate per product

ProductID	Month	Revenue	Prev_Month_Revenue	MoM_Growth_Percent
P201	2023-02	1500	1000	50.0

NOTE:- whenever dividing, please keep the risk of dividing by 0 in mind, and use `nullif(denom, 0)`. the `nullif` checks whether the denominator is 0 or not. if 0, it will convert the denominator into null and answer will be null with no undefined error. DO IT ALWAYS WHEN THERE IS A DENOMINATOR IN THE EXPRESSION.

```
select product_id, month, revenue,

prev_revenue, (revenue-prev_revenue)*100/nullif(prev_revenue,0) as 'growthrate' from

(select *,
```

```
lag(revenue) over(partition by product_id order by month) as 'prev_revenue'

from (SELECT product_id, date_format(order_date,'%Y-%m') as 'month', sum(sales) as
'revenue' FROM cleanedsup

group by product_id, date_format(order_date,'%Y-%m'))t) m

where (revenue-prev_revenue)*100/prev_revenue is not null
```

5. Churned customers per month

Month	Churned_Customers
2023-03	120

Explanation:- (read explanation of q15 first)

same as q15 only, e1 having total customers of current month and e2 having retained customers in next month. but, if you subtract the total customers of e1 with retained customers of e2, you will get those customers in current month which are not in next month. and that is what churned customers are. hence, count after subtraction gives us churned customers.

```
select e1.order_month ,count(distinct(e1.customer_id)) as
'total_cust',count(distinct(e1.customer_id)) - count(distinct(e2.customer_id)) as
'churned_cust',

round((count(distinct(e1.customer_id)) -
count(distinct(e2.customer_id)))*100/count(distinct(e1.customer_id)),2) as 'churn_rate'

from (SELECT DISTINCT

customer_id,

DATE_FORMAT(order_date, '%Y-%m') AS order_month

FROM cleanedsup) e1

left join (select DISTINCT

customer_id,

DATE_FORMAT(order_date, '%Y-%m') AS order_month

FROM cleanedsup) e2

on e1.customer_id = e2.customer_id AND PERIOD_DIFF(REPLACE(e2.order_month, '-', ''),
REPLACE(e1.order_month, '-', '')) = 1

group by e1.order_month
```

order by e1.order_month

6. Top 3 products by sales in each region monthly

Region	Month	ProductID	Total_Sales	Rank
East	2023-04	P302	9000	1

select * from (select *,

dense_rank() over(partition by region, month order by total_sales desc) as 'rank'

from (select region, date_format(order_date, '%Y-%m') as 'month', product_id, sum(sales) as 'total_sales' from cleanedsup

group by region, date_format(order_date, '%Y-%m'), product_id) t)m

where m.rank <=3

7. % contribution of top 3 products to monthly revenue

Month	Total_Revenue	Top3_Revenue	Contribution_Percent
2023-05	100000	52000	52.0

select month, round(total_revenue,2) as 'tot_rev', round(sum(sales),2) as 'top3_rev',
round(sum(sales)*100/total_revenue,2) as 'contribution_percent' from (select *,

sum(sales) over(partition by month) as 'total_revenue',

dense_rank() over(partition by month order by sales desc) as 'rank'

from (select date_format(order_date, '%Y-%m') as 'month', sales

from cleanedsup) t)m

where m.rank <= 3

group by month, total_revenue

8. Customer LTV Bucketing

CustomerID	Total_LTV	LTV_Bucket
C301	70000	High

select customer_id, total_ltv,

case

when parts = 1 then 'low'

when parts = 2 then 'mid'

when parts = 3 then 'high'

```

else 'very high' end as 'ltv_bucket'

from (select distinct customer_id, total_ltv,
ntile(4) over(order by total_ltv) as 'parts'
from (select customer_id,
sum(sales) over(partition by customer_id) as 'total_ltv'
from cleanedsup) t)m

```

9. Time since last order (per order)

CustomerID	OrderID	OrderDate	Days_Since_Last_Order
C101	O123	2023-02-10	30

```

select *,
datediff(order_date, lag_date) as 'days_btw'
from (select customer_id, order_id, order_date,
lag(order_date) over(partition by customer_id order by order_date) as 'lag_date'
from cleanedsup) t
where datediff(order_date, lag_date) is not null

```

10. 3 consecutive weekday orders

CustomerID	Start_Date	Weekdays_Streak
C221	2023-07-03	Mon-Wed

11. First and last order gap (per customer)

CustomerID	First_Order	Last_Order	Days_Between
C231	2022-01-01	2023-01-01	365

```

select customer_id, min(order_date) as 'first_order', max(order_date) as 'last_order',
datediff(max(order_date),min(order_date)) as 'days_between'
from cleanedsup
group by customer_id

```

12. Customers with increasing order value in 3 purchases

CustomerID	Order1_Value	Order2_Value	Order3_Value
C401	1000	1500	2000

this is the one which i did before i leetcode i think, where i had to find 3 consecutive items which are arranged in increasing order.

```
select customer_id, order_date, lag_sales as 'first_order', sales as 'second_order', lead_sales  
as 'third_order' from (select *,  
  
case when sales<lead_sales and sales>lag_sales then 'keep' else null end as 'yn'  
  
from (select customer_id, order_date, sales,  
  
lead(sales) over (partition by customer_id order by customer_id, order_date) as 'lead_sales',  
  
lag(sales) over (partition by customer_id order by customer_id, order_date) as 'lag_sales'  
  
from cleanedsup) t) m  
  
where yn is not null
```

13. Top customers per region per month

Region	Month	Salesperson	Total_Sales
South	2023-03	Alice	25000

```
select region, month, customer_name, total_sales from (select *,  
  
max(total_sales) over(partition by region, month) as 'max_sales'  
  
from (select region, date_format(order_date, '%Y-%m') as 'month', customer_name,  
round(sum(sales),2) as 'total_sales' from cleanedsup  
  
group by region, date_format(order_date, '%Y-%m'), customer_name) t) m  
  
where total_sales = max_sales
```

14. Monthly retention rate

Month	Total_Customers	Retained_Customers	Retention_Rate (%)
2023-04	500	350	70.0

Explanation:-

we firstly take out the distinct order_date and customer_id pairs as a table. now join the same 2 tables using 'left join' operation, on:-

i) obviously the customer_id, because whichever customer satisfies the condition:-

customer_id of current month = customer_id of next month -----> is the customer retained for the current month. also, as it is left join, it keeps all the customers of the current month but only keeps those customers in next month which were in current month. so if you count distinct customers in e1, they will be total customers and for the same in e2, it will be retained customers (those which are in e1 too).

ii) difference between the next month and current month being 1. this will facilitate that the customers of current month are getting matched with the customers of next month only. i was really confused and in period_diff(), i put e1 first and then e2, which is wrong because e1 is current month and e2 is next month, hence we got to do next month - current month right? and NOT current month - next month.

```
select e1.order_month ,count(distinct(e1.customer_id)) as 'total_cust',
count(distinct(e2.customer_id)) as 'retained_cust',

count(distinct(e2.customer_id))*100/count(distinct(e1.customer_id)) as 'retention_rate'

from (SELECT DISTINCT

    customer_id,

    DATE_FORMAT(order_date, '%Y-%m') AS order_month

FROM cleanedsup) e1

left join (select DISTINCT

    customer_id,

    DATE_FORMAT(order_date, '%Y-%m') AS order_month

FROM cleanedsup) e2

on e1.customer_id = e2.customer_id AND PERIOD_DIFF(REPLACE(e2.order_month, '-', ''),
REPLACE(e1.order_month, '-', '')) = 1

group by e1.order_month

order by e1.order_month
```

15. Products with >50% revenue drop MoM

ProductID	Month	Prev_Month_Revenue	Current_Revenue	Drop_Percent
P501	2023-06	20000	8000	60.0

```
select product_id, month, prev_revenue, current_revenue, abs(round((current_revenue -
prev_revenue)*100/nullif(prev_revenue,0),2)) as 'drop' from (select *,

lag(current_revenue) over(partition by product_id order by product_id, month) as
'prev_revenue'

from (select product_id, date_format(order_date, '%Y-%m') as 'month', sum(sales) as
'current_revenue' from cleanedsup

group by product_id, date_format(order_date, '%Y-%m')) t) m
```

where (current_revenue - prev_revenue)*100/prev_revenue < -50

16. Most loyal customers by purchase frequency

CustomerID	Orders	Total_Days	Frequency_Per_30Days
C603	12	180	2.0

```
select customer_id, count(order_id) as 'orders', nullif(datediff(max(order_date),
min(order_date)),0) as 'total_days',
count(order_id)/(datediff(max(order_date), min(order_date))/30) as 'frequency'
from cleanedsup
group by customer_id
```

17. Product categories with 3-month increasing sales

Category	Month1	Month2	Month3	Trend_Confirmed
Furniture	5000	5500	6000	TRUE

```
select * from (select category, lag_sales as 'month1', total_sales as 'month2', lead_sales as
'month3',
case when lag_sales< total_sales and total_sales < lead_sales then 'TRUE' else 'FALSE' end as
'trend'
from (select *,
lag(total_sales) over(partition by category order by category, month) as 'lag_sales',
lead(total_sales) over(partition by category order by category, month) as 'lead_sales'
from (select category, date_format(order_date, '%Y-%m') as 'month', round(sum(sales),2) as
'total_sales' from cleanedsup
group by category, date_format(order_date, '%Y-%m')) t) m) s
where trend = 'TRUE'
```

18. Customers with 1 order per month for past 6 months

CustomerID	Orders_Count	Monthly_Order_Streak
C701	6	TRUE

19. Months where sales peaked 3 years in a row

Category	Month	Year1_Peak	Year2_Peak	Year3_Peak	Peak_Confirmed
Office	March	TRUE	TRUE	TRUE	TRUE


```

select category, month, prev1, prev2, prev3,

case when month = prev1 and prev1 = prev2 and prev2 = prev3 then 'TRUE' else 'FALSE'
end as 'tf'

from (select category, year, month,

lag(month) over(partition by category order by year, month) as 'prev1',
lag(month,2) over(partition by category order by year,month) as 'prev2',
lag(month,3) over(partition by category order by year, month) as 'prev3'

from (select *,

max(monthly_sales) over(partition by category, year) as 'max_sales'

from (select category, date_format(order_date, '%Y') as 'year',

date_format(order_date, '%M') as 'month', round(sum(sales),2) as 'monthly_sales' from
cleanedsup

group by category, date_format(order_date, '%Y'), date_format(order_date, '%M')) t) m

where monthly_sales = max_sales) k

where prev3 is not null

```